arXiv:2611.09197v1 [cs.AI] 6 Nov 2026

Safe Reinforcement Learning via Probabilistic Option Shielding

Gian Luca Ascari*
Apixa
gianlu.ascari@gmail.com

Isidoro Tamassia
KU Leuven
isidoro.tamassia@kuleuven.be

Ying Jiao
KU Leuven
ying.jiao@kuleuven.be

Luc De Raedt
KU Leuven, Örebro University
luc.deraedt@kuleuven.be

Giuseppe Marra
KU Leuven
giuseppe.marra@kuleuven.be

Abstract

Shielding is a safe reinforcement learning technique that provides safety guarantees while learning, by blocking unsafe actions. Traditional shielding relies on a complete model of the environment to provide long-term guarantees. Probabilistic logic shielding (PLS) lifts the requirement of having such model, but only considers safety in the short term, typically looking only one or few steps ahead. We propose Probabilistic Option Shielding (POS), which extends PLS to the option level, recovering long-horizon safety without reintroducing the need for a complete environment model. POS learns a model-free critic directly from one-step PLS signals, to estimate the probability that an entire option will complete safely under uncertainty. We evaluate POS on safe navigation tasks in the LogiCity environment, and show that it achieves considerably safer training while matching the task performance of standard step-level PLS and other safety related reinforcement learning baselines.

1 Introduction

The safety of an agent acting in an environment typically depends not only on the immediate effects of its actions but also on their future consequences. For example, accelerating towards a green traffic light while driving a car may be safe in the moment, but later leave too little distance to stop safely once it becomes red. In reinforcement learning (RL), such consequences can be addressed with options: temporally extended behaviors composed of multiple actions. Our goal is to propose an effective way of enforcing long-term behavioral safety by combining options with probabilistic logic shielding methods.

Shielding provides a principled approach to safety in RL by evaluating candidate actions based on safety knowledge. Classical approaches, rooted in model checking and formal verification [Baier and Katoen, 2008], assume precise safety specifications and rely on a model of the environment to enforce them throughout execution [Alshiekh et al., 2018]. Subsequent work has extended shielding to allow for uncertainty in safety specifications [Carr et al., 2023, Jansen et al., 2020, Yang et al., 2023, Hamel-De le Court et al., 2025]. In particular, Probabilistic Logic Shields (PLS) [Yang et al., 2023] avoid the need for a transition model by encoding safety conditions directly in a probabilistic (logic) program. The program computes action-safety probabilities, which are used to reweight the agent's policy while maintaining the overall process differentiable, and can therefore be integrated

*Work done while at KU Leuven.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Neuro-Symbolic Embodied Intelligence.

into state-of-the-art policy-gradient methods [Schulman et al., 2015, 2017]. However, approaches like PLS do not directly consider the long-term consequences that an action may cause. Although multi-step safety could, in principle, be captured by modeling future transitions within the probabilistic logic program, doing so would reintroduce the need for knowledge of the environment dynamics and substantially increase the cost of inference.

The options framework [Sutton et al., 1999b], employed by most approaches to Hierarchical RL (HRL, [Pateria et al., 2021]), provides a natural level of abstraction for addressing these limitations. Rather than choosing a new low-level action at every step, a high-level policy selects an option and the agent commits to its execution until a termination condition triggers. Shielding at this level requires assigning each option the probability that captures whether its entire execution satisfies the safety specification. Unfortunately, computing this probability on-the-fly also generally requires knowledge of the transition model.

In this work, we show that the safety of an option can be learned without access to a model of the environment dynamics. More specifically, we propose Probabilistic Option Shielding (POS), which learns an option-safety critic by temporal-difference (TD) updates from observed transitions [Sutton and Barto, 2018], either in a pretraining phase or together with the policy. The learned critic is then used to reweigh the option-policy analogously to the PLS action-level shielding, thereby reducing the probability of unsafe behavior.

Contributions.

- We lift PLS to the options framework, extending the short-sighted traditional PLS to long-term behaviors.
- We show that under mild assumptions, the resulting option-safety operator has a unique fixed point and can be optimized efficiently.
- We estimate the safety of options by TD learning with a safety critic, realized either tabularly or as a neural network, and either pretrained or updated online during policy learning. We demonstrate experimentally that this yields considerably safer training at comparable returns to the baselines.
- We release an open-source implementation.[2]

2 Background

2.1 Markov Decision Processes

We consider a discounted Markov decision process (MDP, Puterman [2014]) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \rho_0, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the set of (low-level) actions, $P(s' \mid s, a)$ is the transition probability, $R(s, a, s')$ is the expected one-step reward, ρ_0 is the initial-state distribution, and $\gamma \in [0, 1)$ is the discount factor. At time step t, the agent observes a state s_t, samples an action $a_t \sim \pi(\cdot \mid s_t)$, and the environment samples a successor state according to $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. The interaction therefore produces a trajectory $s_0, a_0, r_0, s_1, a_1, r_1, \ldots$, where $s_0 \sim \rho_0$ and $\mathbb{E}[r_t \mid s_t, a_t, s_{t+1}] = R(s_t, a_t, s_{t+1})$. The objective is to find a policy that maximises the expected discounted return

$$J(\pi) = \mathbb{E}_{\pi,P}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right].$$

2.2 Options and Temporal Abstraction

The options framework extends an MDP with temporally extended actions [Sutton et al., 1999b]. A Markov option is a tuple $o = (\mathcal{I}_o, \pi_o, \beta_o)$ where $\mathcal{I}_o \subseteq \mathcal{S}$ is the initiation set, $\pi_o(a \mid s)$ is the intra-option policy over low-level actions, and $\beta_o(s) \in [0, 1]$ is the probability that the option terminates upon reaching state s. The set of options available in state s is $\mathcal{O}(s) = \{o \in \mathcal{O} : s \in \mathcal{I}_o\}$. When the agent selects option o in state s_t, the option repeatedly samples primitive actions according to $a_t \sim \pi_o(\cdot \mid s_t)$. After arriving in s_{t+1}, the option terminates with probability $\beta_o(s_{t+1})$; otherwise,

[2] https://github.com/gianlucaascari/probabilistic-option-shielding.git

control remains with the same option. Consequently, an option executes for a random duration $T_o \geq 1$ and produces the state-action sequence $s_0, a_0, s_1, \ldots, a_{T_o-1}, s_{T_o}$. An option also terminates if the underlying episode ends.

A high-level policy over options $\Pi(o \mid s)$, $o \in \mathcal{O}(s)$ selects an option whenever the previous one terminates. The selected option then retains control until its own termination condition is satisfied. Thus, decisions made by Π may occur at irregular time intervals. The resulting high-level decision process is also called a semi-Markov decision process (SMDP).

2.3 Probabilistic Logic Shields

Shielding incorporates prior safety knowledge into an agent's decisions by preventing actions that violate a given specification [Alshiekh et al., 2018]. Probabilistic Logic Shields (PLS) extend this idea to specifications involving uncertain facts [Yang et al., 2023]. In PLS, a probabilistic logic program (ProbLog, De Raedt et al. [2007]) encodes the safety specification and, given a state s and candidate primitive action a, computes $\mathbf{P}(\text{safe} \mid s, a) \in [0, 1]$. This quantity represents the probability, according to the encoded specification and a safety-relevant description of the state s, that executing a in s is safe. The probability of each base fact may be provided by a neural network.

For an action policy π, the probability of selecting a safe action in state s is obtained by marginalizing over the policy:

$$\mathbf{P}_\pi(\text{safe} \mid s) = \sum_{a \in \mathcal{A}} \mathbf{P}(\text{safe} \mid s, a)\pi(a \mid s).$$

PLS constructs a shielded policy by conditioning the original policy on safety:

$$\pi^+(a \mid s) = \frac{\mathbf{P}(\text{safe} \mid s, a)\pi(a \mid s)}{\mathbf{P}_\pi(\text{safe} \mid s)},$$

Thus, actions retain their relative probability under π but are also weighted according to their probability of satisfying the safety specification. Because ProbLog programs are differentiable [De Raedt et al., 2007], the shielded policy can be incorporated into any policy-gradient method [Williams, 1992, Sutton et al., 1999a], including popular neural methods such as TRPO [Schulman et al., 2015] and PPO [Schulman et al., 2017], while retaining the underlying learning objective.

As described, PLS evaluates the safety of the next low-level action. Looking multiple steps ahead is possible when future transitions are represented inside the probabilistic logic program, but this may be cumbersome to specify exactly. Moreover, performing multi-step planning increases sharply the cost of inference. These limitations motivate us to apply shielding to temporally extended decisions compactly represented by options.

3 Methodology

We now illustrate our method for temporally extended probabilistic shielding, which we call Probabilistic Option Shielding (POS). POS combines concepts from the neurosymbolic PLS method and the option framework detailed above. We begin by defining the probability of option-level safety and show that it admits a recursive computation which provably converges to a fixed point. Then, we detail how we can learn a safety critic that estimates this quantity in practical settings with temporal-difference (TD) learning techniques [Sutton and Barto, 2018] from one-step safety signals. Note that we assume the option set is given; extending our method by also learning a meaningful set of options from low-level actions is left for future work.

3.1 Option Safety

An option o executes for a random number of steps $T_o \geq 1$. We define its cumulative safety as the expected product of the per-step safety probabilities along its execution:

$$\mathbf{P}(\text{safe} \mid s, o) = \mathbb{E}_{\tau \sim (P, \pi_o, \beta_o, s_0 = s)} \left[\prod_{t=0}^{T_o - 1} \mathbf{P}(\text{safe} \mid s_t, a_t) \right], \tag{1}$$

where $\tau = (s_0, a_0, \ldots, s_{T_o})$ is the trajectory determined by the intra-option policy π_o, the environment dynamics P, and the option-termination probability β_o. Assuming that the per-step safety events

are conditionally independent given the state-action trajectory, the product inside the expectation is the probability that every primitive action along a particular execution of o is safe. The expectation then averages this probability over all possible executions of the option from s.

The high-level shielded policy is obtained by applying the PLS reweighting rule to options:

$$\Pi^{+}(o \mid s) = \frac{\mathbf{P}(\text{safe} \mid s, o)\Pi(o \mid s)}{\mathbf{P}_{\Pi}(\text{safe} \mid s)}, \qquad \mathbf{P}_{\Pi}(\text{safe} \mid s) = \sum_{o \in \mathcal{O}(s)} \mathbf{P}(\text{safe} \mid s, o)\Pi(o \mid s). \tag{2}$$

A primitive action a can be represented as a one-step option o_a whose intra-option policy selects a deterministically. In this case,

$$\mathbf{P}(\text{safe} \mid s, o_a) = \mathbf{P}(\text{safe} \mid s, a),$$

so the proposed formulation recovers action-level shielding as a special case. The remaining challenge is to estimate $\mathbf{P}(\text{safe} \mid s, o)$ without manually specifying the safety of every option.

3.2 Recursive Form and Iterative Convergence

Equation (1) admits a recursive decomposition. Conditioning on the first action a and the resulting state s', the option either terminates at s', in which case no further safety terms are accumulated, or continues from s'. Combining these cases using the termination probability $\beta_o(s')$ gives

$$\mathbf{P}(\text{safe} \mid s, o) = \mathbb{E}_{\substack{a \sim \pi_o(\cdot \mid s) \\ s' \sim P(\cdot \mid s, a)}} \left[\mathbf{P}(\text{safe} \mid s, a) \left(\beta_o(s') + \big(1 - \beta_o(s')\big) \mathbf{P}(\text{safe} \mid s', o) \right) \right]. \tag{3}$$

The continuation probability $1 - \beta_o(s')$ acts as a state-dependent discount. If the option terminates at s', the current action-safety probability is the only contribution. If it continues, this probability is multiplied by the cumulative safety of executing the remainder of the option from s'.

We now show that repeatedly applying Equation (3) converges to a fixed point. This establishes that $\mathbf{P}(\text{safe} \mid s, o)$ is well defined and can be computed by dynamic programming or estimated using methods similar to value-function evaluation [Bellman, 1966]. Let $\mathcal{S}$ be the finite set of states reachable under the option set, and define the safety operator $\mathcal{T}^o$ on $[0,1]^{\mathcal{S}}$ by

$$(\mathcal{T}^o f)(s) = \mathbb{E}_{\substack{a \sim \pi_o(\cdot \mid s) \\ s' \sim P(\cdot \mid s, a)}} \left[\mathbf{P}(\text{safe} \mid s, a) \left(\beta_o(s') + \big(1 - \beta_o(s')\big) f(s') \right) \right]. \tag{4}$$

Assumption 1 (Bounded option duration)**.** *There exists a finite N such that, for every $s \in \mathcal{S}$,*

$$\mathbf{P}(T_o \leq N \mid s_0 = s) = 1,$$

where $T_o \geq 1$ is the number of steps executed by option o before termination.

Assumption 1 requires options to terminate within a uniformly bounded number of steps. It is satisfied, for example, when options have an explicit maximum duration or when they are executed within finite-horizon episodes. Under this assumption, Equation (3) is the fixed-point equation $\mathcal{T}^o f = f$. The proof of the following result is given in Appendix A.

Theorem 1 (Fixed point)**.** *Under Assumption 1, $\mathcal{T}^o$ admits a unique fixed point in $[0,1]^{\mathcal{S}}$. Moreover, for any initial $f \in [0,1]^{\mathcal{S}}$, the sequence*

$$f,\ \mathcal{T}^o f,\ (\mathcal{T}^o)^2 f,\ \ldots$$

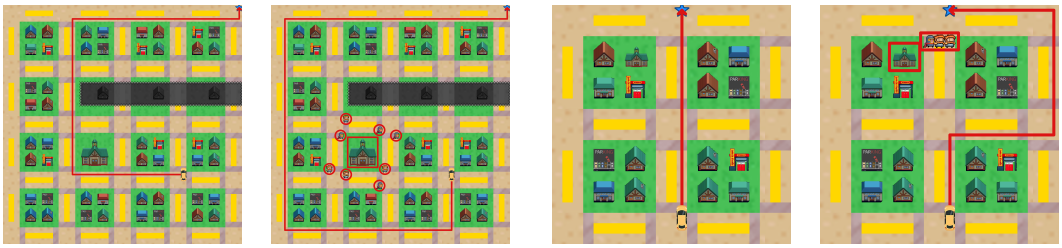
reaches this fixed point after at most N applications of $\mathcal{T}^o$.

3.3 Temporal-Difference Safety Evaluation

Evaluating the expectation in Equation (3) analytically requires the transition model $P(s' \mid s, a)$, which is unavailable in the model-free setting. We therefore estimate it using temporal-difference (TD) learning [Sutton and Barto, 2018]. We maintain a *safety critic* $\hat{P}(\text{safe} \mid s, o)$, initialized in $[0,1]$ for each state-option pair, and update it from observed transitions.

For a transition (s_t, a_t, s_{t+1}) generated during the execution of option o, the bootstrapped TD target is

$$y_t = \mathbf{P}(\text{safe} \mid s_t, a_t) \left[\beta_o(s_{t+1}) + \big(1 - \beta_o(s_{t+1})\big) \hat{P}(\text{safe} \mid s_{t+1}, o) \right]. \tag{5}$$



(a) **Large Map.** `school-time` off (L) and on (R). (b) **Stochastic Map.** `school-time` off (L) and on (R).

Figure 1: Visualizations of the *Large Map* and the *Stochastic Map* and the corresponding safety concerns with `school-time` off or on. Specifically, the black area indicates a military zone, while pedestrians are circled in red. The red arrow is the optimal safe trajectory from the start to the goal.

The critic estimate at s_t is updated with step size $\alpha_n \in [0, 1]$:

$$\hat{P}(\text{safe} \mid s_t, o) \leftarrow \hat{P}(\text{safe} \mid s_t, o) + \alpha_n \left[y_t - \hat{P}(\text{safe} \mid s_t, o) \right]. \tag{6}$$

The immediate term $\mathbf{P}(\text{safe} \mid s_t, a_t)$ is already computed by the PLS shield, so evaluating the target requires no additional probabilistic-logic inference. Because the term in brackets in Equation (5) is a convex combination of 1 and $\hat{P}(\text{safe} \mid s_{t+1}, o)$, the target y_t lies in $[0, 1]$. Equation (6) is itself a convex combination of the current estimate and the target. Consequently, the critic remains in $[0, 1]$ throughout learning, provided that it is initialized in this interval.

When the state space is sufficiently small, the safety critic can be represented as a table with one entry $\hat{P}(\text{safe} \mid s, o)$ for each state-option pair (s, o). After observing a transition generated by executing option o, the corresponding entry is updated toward the TD target using Eq. (6). For larger or continuous state spaces, we can instead approximate the critic with a neural network $\hat{P}_\theta(\text{safe} \mid s, o)$. Its parameters are trained by semi-gradient TD learning [Sutton and Barto, 2018], minimizing the squared error between the current estimate and the bootstrapped target while treating the latter as constant during each update.

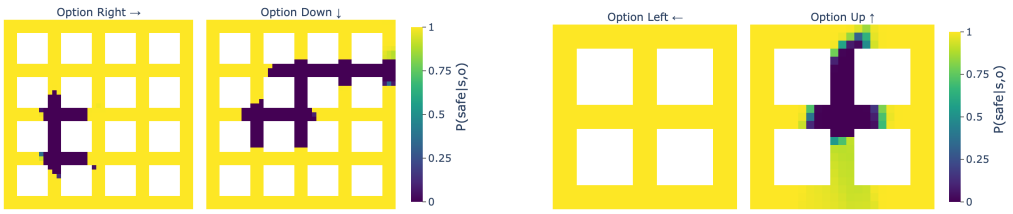
Importantly, the option-safety fixed point does not depend on the option-level policy Π: changing Π only affects which state-option pairs are visited. Consequently, transitions collected under any option-level policy can be used to pretrain the critic. When a separate data-collection phase is impractical, the critic can be learned online using the transitions collected during task-policy learning. We evaluate both the pretrained and online regimes.

4 Experiments

4.1 Setup

Environment. We evaluate our approach in LogiCity [Li et al., 2024], a recently proposed neuro-symbolic AI benchmark. In each episode, the agent starts at a fixed grid cell and must reach a fixed goal by selecting one of five primitive actions: *left*, *right*, *up*, *down*, or *stop*. At each step, the agent observes its current coordinates (x, y) and a binary `school-time` variable. The latter is randomly sampled as $\text{Bernoulli}(0.5)$ at the beginning of each episode and remains fixed throughout the episode. When `school-time` is active, pedestrians occupy cells along the shortest path to the goal, making that route unsafe. An episode terminates when the agent reaches the goal, violates a safety constraint, or reaches a fixed step limit. The agent receives a reward of 1 for reaching the goal and 0 otherwise.

We consider two variants, the *Large Map* (Figure 1a) and the *Stochastic Map* (Figure 1b). The *Large Map* includes pedestrians who only show up during `school-time`, as well as a military zone that is always unsafe (independently of `school-time`). We use this map to evaluate whether the proposed method can enforce both static and context-dependent safety constraints over longer horizons: as shown in Figure 1a, the optimal safe path requires a long detour. The *Stochastic Map* is smaller but introduces a simulated *inertia* effect: at each step, with probability 0.3, the agent is forced to repeat the previously executed low-level action rather than the one it would have selected. Intuitively, this



(a) **Large map.** Options `right` (L) and down (R). (b) **Stochastic map.** Options `left` (L) and up (R).

Figure 2: **Learned option safety.** Examples neural critic estimates for *Large Map* and *Stochastic Map*. Each heatmap shows the estimates for each map position at school time. Color represents the probability of safety, ranging from certainly unsafe (dark) to certainly safe (yellow).

requires the agent to be more far-sighted since trying to brake or change direction too close to the pedestrians may fail due to the inertia.

Method. Safety is specified by a ProbLog program [De Raedt et al., 2007] (see Appendix B.3) that maps the four adjacent-cell sensors and the selected action to $\mathbf{P}(\text{safe} \mid s, a)$, as in the standard PLS setup [Yang et al., 2023]. We take the rules to be correct and the sensor readings to be noise-free. Then, four options are defined, one per cardinal direction. Each navigates from the agent's current cell to the next intersection in that direction by executing the corresponding low-level actions and terminates on arrival, or earlier if the episode ends. The safety of each option is estimated via a safety critic, which is instantiated in a neural form (a comparison with a tabular instantiation is available in Appendix D). The resulting estimates are used to shield the options as described in section 3. As the underlying policy-learning method, we choose PPO [Schulman et al., 2017], a state-of-the art model-free RL algorithm. We test our method (`POS-PPO`) both when the critic is pretrained and when it is learned online alongside the policy. Further details, including the safety program, the option definition, and the critic instantiation can be found in Appendix B.

Baselines and metrics. We compare our approach to five distinct baselines: unshielded `PPO` over low-level actions [Schulman et al., 2017], PPO with standard one-step PLS shielding (`PPO-PLS`, Yang et al. [2023]), unshielded PPO over options (`OptionPPO`), PPO over options with additional reward-shaping (`OptionPPO-RS`), and PPO over options with the safety constraints modeled as in a constrained MDP [Altman, 1999] and enforced with a Lagrangian approach (`OptionPPO-Lag`, Tessler et al., 2019, Ray et al., 2019). Details on the implementations and hyperparameter tuning of each method can be found in Appendix B. For each experiment, we report cumulative safety violations and episodic discounted return averaged over 10 seeds with the corresponding standard error.

Experimental questions. Our experiments aim to address the following questions:

- **Q1.** How accurate are the safety estimates learned by our tabular and neural option-safety critics?
- **Q2.** Does `POS-PPO` yield safer and comparably rewarding policies to baseline approaches?

4.2 Results

(Q1) The critic accurately learns to estimate option safety in both maps. Figure 2 shows the predictions of the pretrained neural critic for some of the options when `school-time` is active. In the *Large Map*, during `school-time`, the safety concerns are avoiding the military zone and avoid driving around the school, both correctly identified by the critic which assigns high probability of a safety violations to the options moving into these areas. In the *Stochastic Map*, the only safety concern is not driving towards the pedestrians at the top of the central road, again correctly identified by the critic which marks as unsafe the option moving up from the center of the map, and as safe going to the left from any position. In Appendix C, we report the complete visualizations including all options and both the `school-time` values; additionally, we report there the safety estimates obtained with a tabular critic.

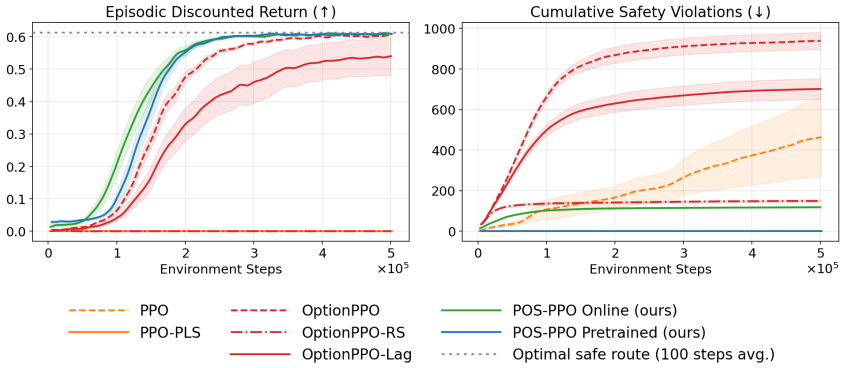


Figure 3: **Performance and safety violations on the large map.** Episodic discounted return (left) and cumulative safety violations (right) are averaged over 10 seeds.

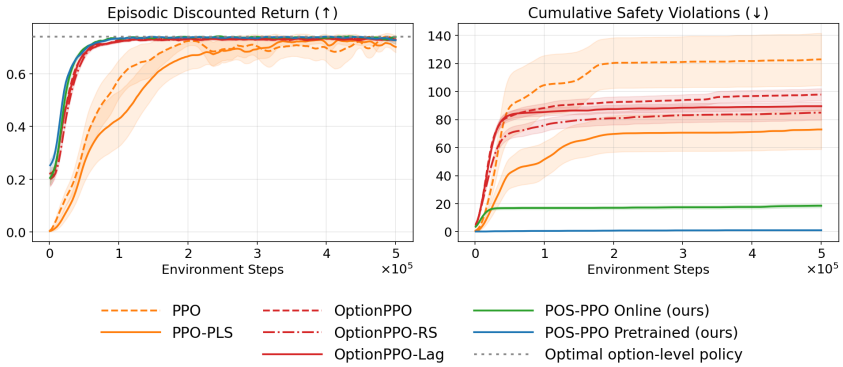


Figure 4: **Performance and safety violations on the stochastic map.** Episodic discounted return (left) and cumulative safety violations (right) are averaged over 10 seeds.

(Q2) `POS-PPO` considerably reduces safety violations compared to the baselines in both maps, while quickly learning an optimal policy. Figure 3 and Figure 4 show the episodic discounted return and cumulative safety violations for the *Large Map* and *Stochastic Map* experiments, respectively. In both cases, `POS-PPO` significantly outperforms the baselines in terms of safety and quickly converges to an optimal policy. In the *Large Map* experiment (Figure 3), `POS-PPO` with a pretrained critic achieves near-zero violations, while learning the critic online reduces the cumulative violations by 87% and 83% with respect to `OptionPPO` and `OptionPPO-Lag`, the only baselines which also consistently solve the task. The other baselines, including `PPO-PLS`, also have few violations but do not learn a policy which solves the task. In the *Stochastic Map* experiment (Figure 4), all the tested algorithms converge to an (almost) optimal policy; `POS-PPO` does so quickly, while significantly reducing violations compared to all the other baselines. The pretrained and online versions reduce violations by 99% and 80% with respect to `OptionPPO-Lag`, and by 99% and 79% with respect to `OptionPPO-RS`.

Moreover, in the *Stochastic Map*, option-level shielding proved significantly safer than its action-level counterpart. While the two are not comparable in the *Large Map*, since the action-level baselines never learned to reach the goal, in this setting both of them are able to reach (near) optimal performance. However, `POS-PPO` incurs considerably fewer violations than PPO-PLS, reducing them by 98% in the pretrained setting and 75% in the online one. The option-level approach shows to be more effective in solving longer-horizon tasks and to be safer under stochastic dynamics.

5 Related Work

Our work is related to several lines of research within Safe Reinforcement Learning (Safe-RL, [García and Fernández, 2015]).

Safe-RL via Shielding. Shielding approaches separate policy optimization from constraint enforcement: a runtime mechanism evaluates the actions proposed by the policy and suppresses, replaces, or reweights actions that may violate a safety specification [Könighofer et al., 2025]. Early work synthesized deterministic shields from temporal-logic specifications, providing formal guarantees whenever the model and specification were accurate [Alshiekh et al., 2018]. Different variations of probabilistic shields extend this idea by comparing actions according to their probability of reaching unsafe states, e.g. using probabilistic model checking [Jansen et al., 2020], MDP-augmentation techniques [Hamel-De le Court et al., 2025], or probabilistic programs [Yang et al., 2023]. Our contribution extends the framework of [Yang et al., 2023] to work naturally over long-horizon safety specifications, therefore inheriting their flexibility and milder MDP specification requirements compared to the other approaches. Model-predictive shielding is another related method which performs online lookahead and invokes a safe backup mechanism when a proposed action cannot be certified [Li and Bastani, 2020, Banerjee et al., 2024]. In contrast, our method learns option-level safety without the need for online planning, a complete dynamics model, or a backup controller. Shielding has also been extended to partial observability [Carr et al., 2023] and multi-agent systems [Elsayed-Aly et al., 2021]; these contributions are orthogonal to our work and could potentially be combined effectively.

Safe Hierarchical RL. Hierarchical reinforcement learning introduces temporal abstractions that divide long-horizon behavior into reusable skills or options [Pateria et al., 2021]. The Option-Critic architecture, for example, learns intra-option policies and termination conditions jointly with a policy over options [Bacon et al., 2017]. Safety-aware extensions address several distinct sources of risk. Safe Option-Critic augments option learning with an uncertainty-sensitive objective that trades expected return against the variability of option outcomes [Jain et al., 2021]. In autonomous driving, hierarchical planning has been used to combine a learned high-level decision maker with low-level controllers designed to preserve safety [Li et al., 2021]. Other approaches formulate hierarchical decision making in terms of expected or percentile-based cost constraints [Lu et al., 2024]. In contrast to these and other similar approaches, we assume a given option set and shield using the learned probability of safe option execution, rather than modifying low-level controllers or optimizing uncertainty-based or cost-based objectives. In this sense, our method is more similar to SQRL [Srinivasan et al., 2020] which also learns a safety critic; however, their critic is at the action level, and their definition of safety assumes terminal unsafe events, which can be unrealistic (for example, a car bumping into another is unsafe but is not generally terminal). Moreover, our approach could be easily integrated with any method that automatically learns options.

6 Conclusions

We introduced a method for learning the safety of temporally extended behaviors using the option framework [Sutton et al., 1999b]; we then showed how these estimates can be effectively integrated as Probabilistic Logic Shields (PLS) during training of any modern policy gradient method with control over options to yield considerably safer training. Our results on the LogiCity simulator showed that our safety critic learns accurate safety estimates, and shielding with it considerably reduces safety violations compared to unshielded methods, standard PLS over actions, reward-shaping, and a common Lagrangian baseline, while still solving the tasks optimally.

Finally, we outline the main limitations of our approach and related directions for future work. First, we assumed a given option set is available to the agent; an interesting extension would be to investigate how to safely learn the options themselves. Additionally, the evaluation focused on variants of the LogiCity environment; a natural next step would be to test our approach on more complex environments, especially more realistic autonomous driving benchmarks such as CARLA [Dosovitskiy et al., 2017].

Acknowledgments and Disclosure of Funding

This research has received funding from the KU Leuven Research Funds (C14/24/092) and from the Flemish Government under the "Onderzoeksprogramma Artificiële Intelligentie (AI) Vlaanderen" programme. Y. Jiao is supported by the EU Framework Program for Research and Innovation Horizon under the Grant Agreement No 101073307 (MSCA-DN LeMuR). L. De Raedt is also supported

by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (Grant agreement No. 101142702) and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 2669–2678, 2018. doi: 10.1609/aaai.v32i1.11797.

Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, pages 1726–1734, 2017. doi: 10.1609/aaai.v31i1.10916.

Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT press Cambridge, 2008.

Arko Banerjee, Kia Rahmani, Joydeep Biswas, and Isil Dillig. Dynamic model predictive shielding for provably safe reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 37, pages 100131–100159. Curran Associates, Inc., 2024. doi: 10.52202/079017-3177.

Richard Bellman. Dynamic programming. *science*, 153(3731):34–37, 1966.

Steven Carr, Nils Jansen, Sebastian Junges, and Ufuk Topcu. Safe reinforcement learning via shielding under partial observability. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 14748–14756, 2023. doi: 10.1609/aaai.v37i12.26723.

Luc De Raedt, Angelika Kimmig, and Hannu Toivonen. ProbLog: A probabilistic Prolog and its application in link discovery. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pages 2462–2467, 2007. URL https://www.ijcai.org/Proceedings/07/Papers/396.pdf.

Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. In *Conference on robot learning*, pages 1–16. PMLR, 2017.

Ingy Elsayed-Aly, Suda Bharadwaj, Christopher Amato, Rüdiger Ehlers, Ufuk Topcu, and Lu Feng. Safe multi-agent reinforcement learning via shielding. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, pages 483–491. International Foundation for Autonomous Agents and Multiagent Systems, 2021. URL https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p483.pdf.

Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015. URL https://jmlr.org/papers/v16/garcia15a.html.

Edwin Hamel-De le Court, Francesco Belardinelli, and Alexander W. Goodall. Probabilistic shielding for safe reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 16091–16099, 2025. doi: 10.1609/aaai.v39i15.33767.

Arushi Jain, Khimya Khetarpal, and Doina Precup. Safe option-critic: Learning safety in the option-critic architecture. *The Knowledge Engineering Review*, 36:e4, 2021. doi: 10.1017/S0269888921000035.

Nils Jansen, Bettina Könighofer, Sebastian Junges, Alex Serban, and Roderick Bloem. Safe reinforcement learning using probabilistic shields. In *31st International Conference on Concurrency Theory (CONCUR 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Bettina Könighofer, Roderick Bloem, Nils Jansen, Sebastian Junges, and Stefan Pranger. Shields for safe reinforcement learning. *Communications of the ACM*, 68(11):80–90, 2025. doi: 10.1145/3715958.

Bowen Li, Zhaoyu Li, Qiwei Du, Jinqi Luo, Wenshan Wang, Yaqi Xie, Simon Stepputtis, Chen Wang, Katia Sycara, Pradeep Ravikumar, et al. Logicity: Advancing neuro-symbolic ai with abstract urban simulation. *Advances in Neural Information Processing Systems*, 37:69840–69864, 2024.

Jinning Li, Liting Sun, Jianyu Chen, Masayoshi Tomizuka, and Wei Zhan. A safe hierarchical planning framework for complex driving scenarios based on reinforcement learning. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2660–2666, 2021. doi: 10.1109/ICRA48506.2021.9561195.

Shuo Li and Osbert Bastani. Robust model predictive shielding for safe reinforcement learning with stochastic dynamics. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7166–7172, 2020. doi: 10.1109/ICRA40945.2020.9196867.

Yuxiao Lu, Arunesh Sinha, and Pradeep Varakantham. Handling long and richly constrained tasks through constrained hierarchical reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 21368–21377, 2024. doi: 10.1609/aaai.v38i19.30132.

Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 54(5):1–35, 2021.

Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. Technical report, OpenAI, 2019.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. Pmlr, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Krishnan Srinivasan, Benjamin Eysenbach, Sehoon Ha, Jie Tan, and Chelsea Finn. Learning to be safe: Deep rl with a safety critic. *arXiv preprint arXiv:2010.14603*, 2020.

Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999a.

Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999b.

Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. Reward constrained policy optimization. In *International Conference on Learning Representations (ICLR)*, 2019.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

Wen-Chi Yang, Giuseppe Marra, Gavin Rens, and Luc De Raedt. Safe reinforcement learning via probabilistic logic shields. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pages 5739–5749, 2023. doi: 10.24963/ijcai.2023/637. URL `https://www.ijcai.org/proceedings/2023/637`.

A Proof of Theorem 1

We measure distances between candidate functions using the supremum norm, $\|f - g\|_\infty = \sup_{s \in \mathcal{S}} |f(s) - g(s)|$. Then, the following lemma bounds the difference between two candidate functions after k applications of the operator $\mathcal{T}^o$ by the probability that the option remains active after k steps.

Lemma 1 (Trajectory unrolling bound)**.** *For any $k \geq 0$ and any $f, g \in [0,1]^{\mathcal{S}}$,*

$$\left\|(\mathcal{T}^o)^k f - (\mathcal{T}^o)^k g\right\|_\infty \leq \sup_{s \in \mathcal{S}} \mathbf{P}(T_o > k \mid s_0 = s) \|f - g\|_\infty.$$

Proof. We prove by induction on k that, for every $s \in \mathcal{S}$,

$$\left|[(\mathcal{T}^o)^k f](s) - [(\mathcal{T}^o)^k g](s)\right| \leq \mathbf{P}(T_o > k \mid s_0 = s) \|f - g\|_\infty. \tag{7}$$

The result then follows by taking the supremum over s.

For $k = 0$, $(\mathcal{T}^o)^0$ is the identity and $|f(s) - g(s)| \leq \|f - g\|_\infty$. Because $T_o \geq 1$, we have $\mathbf{P}(T_o > 0 \mid s_0 = s) = 1$, so the base case holds.

Suppose that Equation (7) holds at k for every state. Applying $\mathcal{T}^o$ once again and subtracting causes the termination terms $\beta_o(s')$ to cancel:

$$\begin{aligned}
&\left|[(\mathcal{T}^o)^{k+1} f](s) - [(\mathcal{T}^o)^{k+1} g](s)\right| \\
&= \left| \mathbb{E}_{\substack{a \sim \pi_o(\cdot|s) \\ s' \sim P(\cdot|s,a)}} \left[\mathbf{P}(\text{safe} \mid s, a)\big(1 - \beta_o(s')\big) \left([(\mathcal{T}^o)^k f](s') - [(\mathcal{T}^o)^k g](s') \right) \right] \right| \\
&\leq \mathbb{E}_{\substack{a \sim \pi_o(\cdot|s) \\ s' \sim P(\cdot|s,a)}} \left[\mathbf{P}(\text{safe} \mid s, a)\big(1 - \beta_o(s')\big) \left| [(\mathcal{T}^o)^k f](s') - [(\mathcal{T}^o)^k g](s') \right| \right] \\
&\leq \mathbb{E}_{\substack{a \sim \pi_o(\cdot|s) \\ s' \sim P(\cdot|s,a)}} \left[\mathbf{P}(\text{safe} \mid s, a)\big(1 - \beta_o(s')\big) \mathbf{P}(T_o > k \mid s_0 = s') \right] \|f - g\|_\infty \\
&\leq \mathbb{E}_{\substack{a \sim \pi_o(\cdot|s) \\ s' \sim P(\cdot|s,a)}} \left[\big(1 - \beta_o(s')\big) \mathbf{P}(T_o > k \mid s_0 = s') \right] \|f - g\|_\infty \\
&= \mathbf{P}(T_o > k + 1 \mid s_0 = s) \|f - g\|_\infty.
\end{aligned}$$

The first inequality follows from $|\mathbb{E}[X]| \leq \mathbb{E}[|X|]$. The second applies the inductive hypothesis, and the third follows since $\mathbf{P}(\text{safe} \mid s, a) \leq 1$. The last equality holds since the option remains active for more than $k + 1$ steps precisely when it does not terminate after the first transition and, from s', its remaining execution lasts for more than k steps. This completes the induction. □

Proof of Theorem 1. We now need to prove that repeated applications of the operator $\mathcal{T}^o$ reach a unique fixed point.

Range preservation. We first verify that $\mathcal{T}^o$ maps $[0,1]^{\mathcal{S}}$ into itself. For any $f \in [0,1]^{\mathcal{S}}$, $\beta_o(s') + \big(1 - \beta_o(s')\big) f(s')$ is a convex combination of 1 and $f(s')$ and therefore lies in $[0,1]$. Multiplying it by $\mathbf{P}(\text{safe} \mid s, a) \in [0,1]$ preserves this range, as does taking the expectation over a and s'. Hence, $\mathcal{T}^o f \in [0,1]^{\mathcal{S}}$.

By Assumption 1, $\mathbf{P}(T_o > N \mid s_0 = s) = 0$ for every $s \in \mathcal{S}$. Setting $k = N$ in Lemma 1 therefore gives $\left\|(\mathcal{T}^o)^N f - (\mathcal{T}^o)^N g\right\|_\infty = 0$ for every $f, g \in [0,1]^{\mathcal{S}}$. Thus, $(\mathcal{T}^o)^N$ is a constant map: there exists some $c \in [0,1]^{\mathcal{S}}$ such that $(\mathcal{T}^o)^N f = c$ for every $f \in [0,1]^{\mathcal{S}}$.

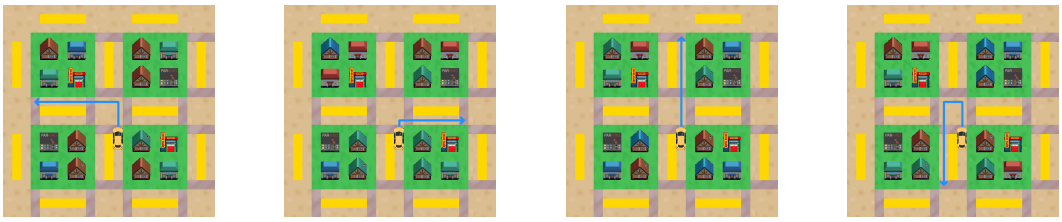
Existence. In particular, $(\mathcal{T}^o)^N c = c$. Then, applying $\mathcal{T}^o$ gives $(\mathcal{T}^o)^{N+1} c = \mathcal{T}^o c$. On the other hand, range preservation implies $\mathcal{T}^o c \in [0,1]^{\mathcal{S}}$, and $(\mathcal{T}^o)^N$ being a constant map gives $(\mathcal{T}^o)^{N+1} c = (\mathcal{T}^o)^N (\mathcal{T}^o c) = c$. Therefore, $\mathcal{T}^o c = c$, so c is a fixed point.

Uniqueness. Suppose that $f^\star$ is another fixed point of $\mathcal{T}^o$. Then $(\mathcal{T}^o)^N f^\star = f^\star$. Because $(\mathcal{T}^o)^N$ maps every element of $[0,1]^{\mathcal{S}}$ to c, it follows that $f^\star = c$. Hence, the fixed point is unique.

Convergence. For any initial $f_0 \in [0,1]^{\mathcal{S}}$, $(\mathcal{T}^o)^N f_0 = c$. Because c is a fixed point, every subsequent iterate also equals c. Thus, iteratively applying of $\mathcal{T}^o$ reaches the unique fixed point after at most N applications. □

Table 1: **Environment parameters.** Positions are given as (row, column) on the grid. All other environment settings are identical between the two maps.

	Large Map	*Stochastic Map*
Grid size	43×43	23×23
Start cell	$(30, 32)$	$(20, 12)$
Goal cell	$(0, 42)$	$(0, 12)$
Step limit	800	250
Pedestrians	8	3
Military zone	rows 14–17, cols 14–42	none
Inertia probability	none	0.3
Discount γ	0.995	0.99



(a) *left* (b) *right* (c) *up* (d) *down*

Figure 5: **The four options.** Each panel traces the trajectory followed from a common starting cell, drawn schematically on a small grid.

B Experimental details

B.1 Environment

The environment used for the experiments is implemented in LogiCity [Li et al., 2024] (MIT License). Table 1 collects the parameters that differ between the two maps we tested.

B.2 Options

The options are assumed to be given in this work. Figure 5 shows the available ones: each one takes the agent from its current position to the next intersection in one of the four cardinal directions. They can be selected in any state, and terminate once they reach the target intersection, or immediately if no further intersection in the chosen direction is present.

B.3 Safety specification

Listing 1 gives the ProbLog program used on the *Large Map*. It maps four adjacent-cell pedestrian sensors, four military-zone sensors and the selected action to $\mathbf{P}(\text{safe} \mid s, a)$. The program reads as follows: the first block shows for each of the five actions the probability the policy assigns to it, and the next two blocks show the corresponding sensor readings in each of the four directions for the pedestrian and military-zone atoms; the `transition` facts map an action to the cell it moves into. The two `unsafe_next` rules then state that an action is unsafe if it brings the agent into a cell holding a pedestrian or belonging to the military zone. Finally, *stop* is safe by construction and any other action is safe when it is not unsafe. Querying `safe_next` yields $\mathbf{P}(\text{safe} \mid s, a)$.

The *Stochastic Map* uses the same program with the four `military_area` sensors and the second `unsafe_next` clause removed, since that map has no military zone.

B.4 PPO hyperparameters

Every presented method trains a PPO policy and they all use the configuration in Table 2. Most hyperparameters are shared across all methods, although between action-level and option-level

Listing 1: **Safety specification for the *Large Map*.**

```
action(0)::action(left).
action(1)::action(right).
action(2)::action(up).
action(3)::action(down).
action(4)::action(stop).

% Other-agent sensors (indices 0-3)
sensor_value(0)::other_agent(left).
sensor_value(1)::other_agent(right).
sensor_value(2)::other_agent(up).
sensor_value(3)::other_agent(down).

% Military-area sensors (indices 4-7)
sensor_value(4)::military_area(left).
sensor_value(5)::military_area(right).
sensor_value(6)::military_area(up).
sensor_value(7)::military_area(down).

transition(left,left).
transition(right,right).
transition(up,up).
transition(down,down).

% Unsafe if the next cell is occupied by another agent
unsafe_next :- action(A), transition(A,NextPos), other_agent(NextPos).
% Unsafe if the next cell falls in the military zone
unsafe_next :- action(A), transition(A,NextPos), military_area(NextPos).

safe_next :- action(stop).
safe_next :- \+ unsafe_next.
```

Table 2: **PPO hyperparameters.** *Opt* and *act* denote the option-level and action-level arms. Rollout length is measured in buffer entries, which are options at the option level and primitive actions at the action level.

	Large Map		*Stochastic Map*	
Hyperparameter	Opt	Act	Opt	Act
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Discount γ	0.995	0.995	0.99	0.99
Clip range	0.15	0.15	0.15	0.15
Value-function coef.	0.5	0.5	0.5	0.5
Entropy coef.	0.01	0.01	0.01	0.01
Epochs per update	8	8	8	8
Rollout length	512	2048	128	512
Minibatch size	128	512	32	128

baselines the rollout length and the minibatch size differ. These are chosen so that the two levels consume comparable numbers of environment steps per update.

B.5 Safety critic

Tabular critic. The tabular critic stores one entry per $(row, col, \texttt{school-time}, o)$ tuple, initialised at 0.5 everywhere in both training regimes. In the pretrained regime the step size starts at $\alpha = 0.1$ and is halved whenever the updates plateau, with training stopped once the entries stabilise across a full pass. In the online regime the step size is fixed at $\alpha = 1$ on the *Stochastic Map* and reduced to $\alpha = 0.99$ on the *Large Map*.

Table 3: **Neural safety-critic settings.**

	Large Map	*Stochastic Map*
Hidden layers	$64, 64$	32
Output power k	4	2
Learning rate	5×10^{-4}	2×10^{-3}
Gradient steps per decision	2	8
Replay-buffer size	5×10^5	5×10^5
Minibatch size	128	128
Optimizer	Adam	Adam
Polyak coefficient τ	0.005	0.005
Online update frequency	Every option decision	Every option decision

Table 4: **Reward-shaping penalty sweep.** Episodic discounted return and cumulative violations at the end of training, mean $\pm$ SEM over five seeds per setting. Selected values in bold

	Large Map		*Stochastic Map*	
Penalty	Return	Violations	Return	Violations
-0.1	0.000 ± 0.000	139.9 ± 2.6	0.716 ± 0.009	112.0 ± 2.3
-0.25	0.000 ± 0.000	144.4 ± 5.8	0.693 ± 0.016	110.7 ± 4.7
-0.5	0.000 ± 0.000	146.9 ± 2.7	0.681 ± 0.016	110.0 ± 8.0
-0.75	$\mathbf{0.000 \pm 0.000}$	$\mathbf{139.4 \pm 2.4}$	0.704 ± 0.007	104.3 ± 2.6
-0.9	0.000 ± 0.000	146.7 ± 1.8	$\mathbf{0.702 \pm 0.004}$	$\mathbf{94.1 \pm 4.9}$
-1.0	0.000 ± 0.000	144.5 ± 10.4	0.679 ± 0.015	98.6 ± 9.2

Neural critic. The neural critic is an MLP that takes the three-dimensional observation as input and produces one safety estimate per option. Because cumulative option-safety probabilities can be very small, we parameterize each output as $\hat{P}(\text{safe} \mid s, o) = \sigma(z_o)^k$, where k is a fixed positive integer. This transformation changes the parameterization of the critic but not the quantity it estimates, since the powered output is trained directly against the TD target in probability space. It allows small probabilities to be represented with less extreme logits: for example, an output of 10^{-6} requires $z_o \approx -13.8$ when $k = 1$, but only $z_o \approx -3.4$ when $k = 4$. We minimize the mean squared TD error using Adam [Kingma and Ba, 2014] with minibatches of size 128. A target network, updated by Polyak averaging with $\tau = 0.005$, provides the bootstrap estimates. Table 3 gives the per-map settings.

B.6 Baselines and hyperparameter selection

`PPO` and `PPO-PLS` operate on low-level actions, the latter applying the one-step shield of Yang et al. [2023] to the program of subsection B.3. `OptionPPO` is the same PPO applied to the policy over options with no safety mechanism. For these baselines, the only hyperparameters are the standard PPO ones reported in Table 2.

Reward shaping. `OptionPPO-RS` adds a constant negative reward for each violated task rule. The penalty was swept over -0.1, -0.25, -0.5, -0.75, -0.9 and -1.0 with the PPO configuration held fixed; Table 4 reports the outcome. For both maps the penalty that held the least violations was selected.

Lagrangian. `OptionPPO-Lag` treats the safety specification as in a constrained MDP, considering a cost of 1 for each safety violation. The sweep values for the cost limit d and the multiplier step size are reported in Table 5. For both maps the selected hyperparameters are the ones yielding the least violations among the ones solving the task.

Table 5: **Lagrangian sweep.** Each cell gives episodic discounted return (top) and cumulative safety violations (bottom) at the end of training, as mean ± standard error over three seeds.

	Multiplier step size			
Cost limit d	0.01	0.02	0.05	0.15
Large Map				
0	0.607 ± 0.002 817 ± 57.0	0.405 ± 0.165 638 ± 63.3	0.203 ± 0.165 370 ± 97.2	0.000 ± 0.000 206 ± 17.2
0.05	0.608 ± 0.002 828 ± 39.1	$\mathbf{0.602 \pm 0.004}$ $\mathbf{773 \pm 32.4}$	0.094 ± 0.076 339 ± 54.5	0.000 ± 0.000 211 ± 16.4
0.15	0.609 ± 0.003 864 ± 11.3	0.408 ± 0.167 593 ± 76.3	0.101 ± 0.083 329 ± 36.9	0.000 ± 0.000 270 ± 11.0
0.3	0.610 ± 0.003 835 ± 5.3	0.606 ± 0.002 832 ± 49.9	0.402 ± 0.164 481 ± 97.9	0.164 ± 0.068 350 ± 43.9
Stochastic Map				
0	0.741 ± 0.003 88 ± 4.5	0.734 ± 0.006 90 ± 9.2	0.731 ± 0.008 95 ± 2.3	0.703 ± 0.019 86 ± 7.7
0.02	0.735 ± 0.006 93 ± 6.2	0.735 ± 0.007 91 ± 2.7	$\mathbf{0.717 \pm 0.004}$ $\mathbf{77 \pm 2.9}$	0.739 ± 0.004 93 ± 2.2
0.05	0.735 ± 0.003 92 ± 0.5	0.739 ± 0.005 87 ± 3.1	0.733 ± 0.004 83 ± 2.1	0.738 ± 0.006 82 ± 5.0
0.15	0.733 ± 0.008 95 ± 2.4	0.745 ± 0.005 95 ± 5.8	0.745 ± 0.001 80 ± 3.6	0.736 ± 0.004 86 ± 2.8

Table 6: **Training time per method.** *Time* is wall-clock seconds per 10^5 environment steps, the median over three sequential runs.

Method	*Large*	*Stochastic*
`PPO`	99.6	105.9
`PPO-PLS`	181.7	126.2
`OptionPPO`	114.8	109.1
`OptionPPO-RS`	95.2	108.3
`OptionPPO-Lag`	114.9	111.8
`POS-PPO-P` (tabular)	100.3	110.6
`POS-PPO-O` (tabular)	143.1	137.7
`POS-PPO-P` (neural)	103.7	110.8
`POS-PPO-O` (neural)	155.7	193.2

B.7 Evaluation protocol

Every experiment is run for 5×10^5 environment steps with ten seeds, and all curves show the mean across seeds with the standard error of the mean. Episodic discounted return is accumulated over each episode, so an episode reaching the goal after L steps scores exactly γ^{L-1}. Cumulative safety violations are the running total of violated safety specifications over the whole run.

B.8 Compute

All experiments were run on a single Apple M1 Pro laptop with 10 CPU cores and 16 GB of memory, under macOS 15.7, with Python 3.11, PyTorch 2.9 and Stable-Baselines3 2.7. Table 6 reports the time (in seconds) needed for each method to reach $100k$ environment steps. The table shows that all methods have comparable training times.

C Additional safety visualizations

The figures below show the full visualizations of the learned critics in both tabular and neural instantiation. Figure 6 and Figure 7 for the *Large Map*, and Figure 8 and Figure 9 for the *Inertia Map*.

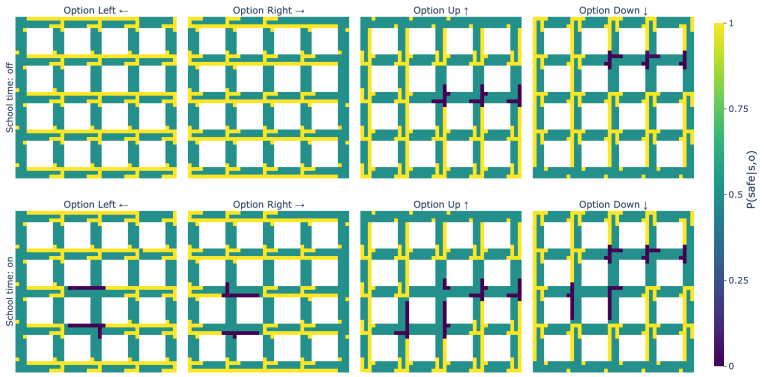


Figure 6: **Pretrained Tabular Critic, Large Map.** The tabular critic estimates the safety of each option in each state. The bottom row is for school time, the top one for no school time. The representation matches Figure 2a

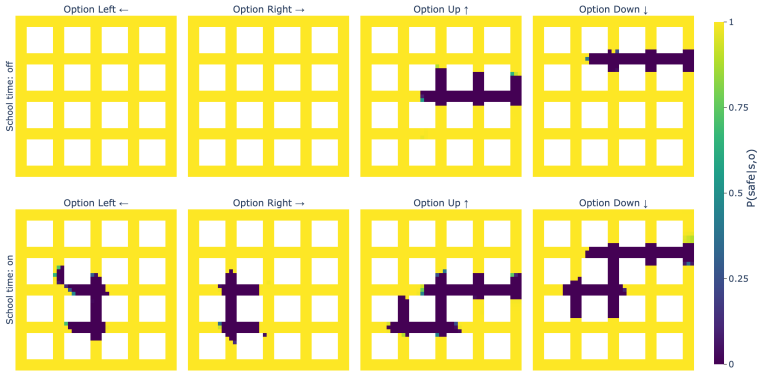


Figure 7: **Pretrained Neural Critic, Large Map.** The neural critic estimates the safety of each option in each state. The bottom row is for school time, the top one for no school time. The representation matches Figure 2a

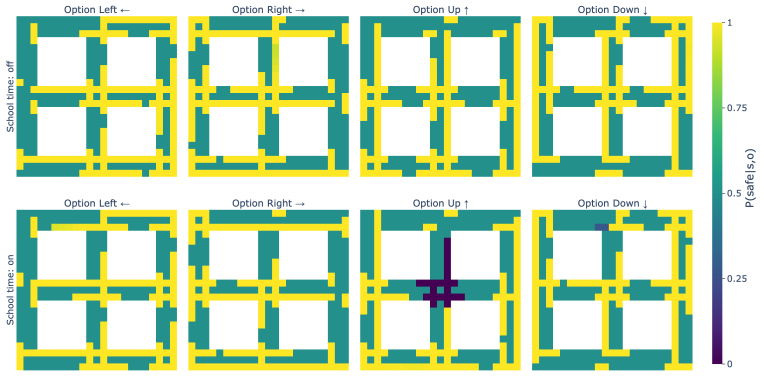


Figure 8: **Pretrained Tabular Critic, Stochastic Map.** The tabular critic estimates the safety of each option in each state. The bottom row is for school time, the top one for no school time. The representation matches Figure 2b

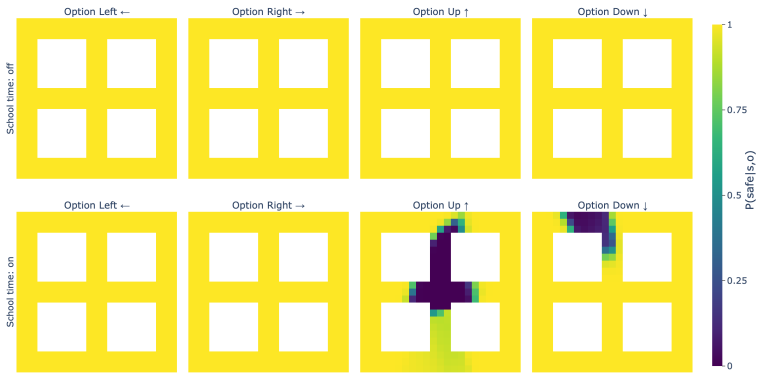


Figure 9: **Pretrained Neural Critic, Stochastic Map.** The neural critic estimates the safety of each option in each state. The bottom row is for school time, the top one for no school time. The representation matches Figure 2b

D Tabular vs Neural Critic Comparison

Figure 10 and Figure 11 show the differences in performance and safety violations of the tabular and neural instantiations of the critic.

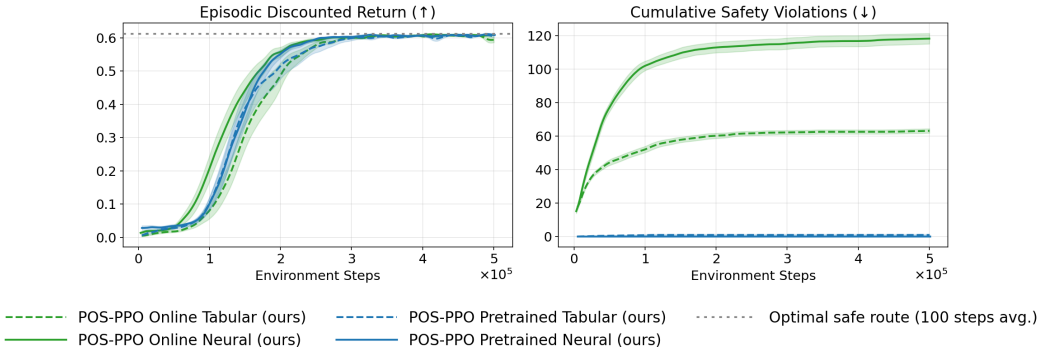


Figure 10: **Performance and Safety Violations Large Map, Tabular vs Neural Critic.** Episodic discounted return (left) and cumulative safety violations (right) averaged over 10 seeds.

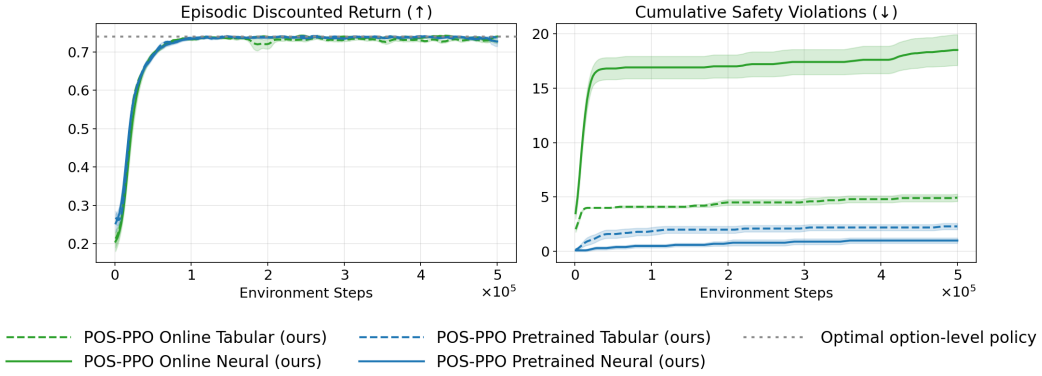


Figure 11: **Performance and Safety Violations Large Map, Tabular vs Neural Critic.** Episodic discounted return (left) and cumulative safety violations (right) averaged over 10 seeds.